*** More Terra Scripts ***

1. The Global Provider & State (The "Skeleton")

Every project starts here. This script sets up the AWS provider and a remote backend (S3 + DynamoDB) to prevent state file corruption.

```
# Standard AWS Provider Configuration
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0" # Lock to a major version for stability
    }
  }
  # Recommended: Store state in S3 for team collaboration
  backend "s3" {
    bucket         = "my-terraform-state-ssdn"
    key            = "global/s3/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-locks" # Prevents concurrent runs
    encrypt        = true
  }
}

provider "aws" {
  region = "us-east-1"
}
```

2. Modular VPC (The "Foundation")

Instead of writing 100 lines of VPC code use the **Official Terraform AWS VPC Module**. It's the industry standard.

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"

  name = "lab-vpc"
  cidr = "10.0.0.0/16"

  # High Availability: Spread across 3 Availability Zones
  azs             = ["us-east-1a", "us-east-1b", "us-east-1c"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24", "10.0.103.0/24"]

  enable_nat_gateway = true # Allows private instances to access internet
  single_nat_gateway = true # Cost-saving for lab environments
}
```

3. EC2 Instance with IAM Role (The "Compute")

This script demonstrates how to give an EC2 instance permissions (e.g., to read from S3) without hard-coding keys.

```
# Create the IAM Role for the Instance
resource "aws_iam_role" "ec2_role" {
  name = "ec2_s3_readonly_role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{ Action = "sts:AssumeRole", Effect = "Allow", Principal =
{ Service = "ec2.amazonaws.com" } }]
  })
}

# Attach S3 Read-Only Policy
resource "aws_iam_role_policy_attachment" "s3_read" {
  role       = aws_iam_role.ec2_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"
}

# Instance Profile (Container for the role)
resource "aws_iam_instance_profile" "profile" {
  role = aws_iam_role.ec2_role.name
}

resource "aws_instance" "app_server" {
  ami           = "ami-0c55b159cbfafe1f0" # Amazon Linux 2
  instance_type = "t3.micro"
  iam_instance_profile = aws_iam_instance_profile.profile.name
  tags = { Name = "Lateral-Lab-Server" }
}
```

4. S3 Bucket with Security Guardrails

Demonstrates encryption and public access blocking.

```
resource "aws_s3_bucket" "secure_data" {
  bucket = "ssdn-labs-encrypted-data-${random_id.id.hex}"
}

# Block all public access (Safety first!)
resource "aws_s3_bucket_public_access_block" "block" {
  bucket = aws_s3_bucket.secure_data.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

# Enable AES-256 Encryption by default
resource "aws_s3_bucket_server_side_encryption_configuration" "encrypt" {
  bucket = aws_s3_bucket.secure_data.id
  rule { apply_server_side_encryption_by_default { sse_algorithm = "AES256"
} }
}
```

5. Application Load Balancer (ALB)

The "Front Door" for microservices.

```
resource "aws_lb" "main" {
  name               = "app-lb"
  internal           = false
  load_balancer_type = "application"
  security_groups    = [aws_security_group.lb_sg.id]
  subnets            = module.vpc.public_subnets
}

resource "aws_lb_target_group" "app_tg" {
  name     = "app-target-group"
  port     = 80
  protocol = "HTTP"
  vpc_id   = module.vpc.vpc_id
}
```

Category: Databases & Caching (Scripts 6–10)

6. RDS Aurora Serverless v2 Cluster

```
resource "aws_rds_cluster" "aurora" {
  cluster_identifier      = "ssdn-aurora-cluster"
  engine                  = "aurora-mysql"
  engine_mode             = "provisioned" # Required for v2 scaling
  engine_version          = "8.0.mysql_aurora.3.04.0"
  database_name           = "trainingdb"
  master_username         = "admin"
  master_password         = "SecurePassword123!" # In Prod: use Secrets
Manager
  skip_final_snapshot     = true

  serverless_v2_scaling_configuration {
    max_capacity = 16.0
    min_capacity = 0.5
  }
}

resource "aws_rds_cluster_instance" "instance1" {
  cluster_identifier = aws_rds_cluster.aurora.id
  instance_class     = "db.serverless"
  engine             = aws_rds_cluster.aurora.engine
  engine_version     = aws_rds_cluster.aurora.engine_version
}
```

7. DynamoDB with Global Secondary Index (GSI)

```
resource "aws_dynamodb_table" "users" {
  name           = "UsersTable"
  billing_mode   = "PAY_PER_REQUEST" # Serverless billing
  hash_key       = "UserID"

  attribute { name = "UserID"; type = "S" }
```

```
  attribute { name = "Email"; type = "S" }

  # GSI allows searching by Email even if UserID is the primary key
  global_secondary_index {
    name            = "EmailIndex"
    hash_key        = "Email"
    projection_type = "ALL"
  }
}
```

8. ElastiCache Redis Replication Group (Multi-AZ)

Production standard: Always use a Replication Group for failover capability.

```
resource "aws_elasticache_replication_group" "redis" {
  replication_group_id          = "prod-redis"
  description                   = "High Availability Redis"
  node_type                     = "cache.t3.micro"
  num_cache_clusters            = 2 # 1 Primary, 1 Replica
  automatic_failover_enabled    = true
  multi_az_enabled              = true
  port                          = 6379
  subnet_group_name             =
aws_elasticache_subnet_group.redis_subnets.name
}
```

Category: Security & Secret Management (Scripts 11–15)

11. KMS Key with Cross-Service Policy

```
resource "aws_kms_key" "app_key" {
  description             = "KMS key for encrypting app secrets"
  deletion_window_in_days = 7
  enable_key_rotation     = true # Best Practice: Automatic rotation
}

resource "aws_kms_alias" "app_key_alias" {
  name          = "alias/app-encryption-key"
  target_key_id = aws_kms_key.app_key.key_id
}
```

12. Secrets Manager with Automatic Rotation (Concept)

```
resource "aws_secretsmanager_secret" "db_password" {
  name        = "prod/db/master_password"
  kms_key_id  = aws_kms_key.app_key.arn
}

resource "aws_secretsmanager_secret_version" "initial_val" {
  secret_id     = aws_secretsmanager_secret.db_password.id
  secret_string = jsonencode({
    username = "admin"
    password = "SuperSecretPassword123!"
  })
```

```
}
```

Category: App Integration & Messaging (Scripts 16–20)

16. SQS Queue with Dead Letter Queue (DLQ)

```
# The "Hospital" for failed messages
resource "aws_sqs_queue" "app_queue_dlq" {
  name = "orders-queue-deadletter"
}

resource "aws_sqs_queue" "app_queue" {
  name = "orders-queue"
  redrive_policy = jsonencode({
    deadLetterTargetArn = aws_sqs_queue.app_queue_dlq.arn
    maxReceiveCount     = 5 # Retry 5 times before moving to DLQ
  })
}
```

17. SNS Topic with Email Subscription

```
resource "aws_sns_topic" "alerts" {
  name = "system-alerts-topic"
}

resource "aws_sns_topic_subscription" "email_alert" {
  topic_arn = aws_sns_topic.alerts.arn
  protocol  = "email"
  endpoint  = "admin@example.com" # Requires manual confirmation via email
link
}
```

Category: Compute & Orchestration (Scripts 21–25)

21. ECS Fargate Task Definition

```
resource "aws_ecs_task_definition" "web_app" {
  family                   = "frontend-task"
  network_mode             = "awsvpc"
  requires_compatibilities = ["FARGATE"]
  cpu                      = "256"
  memory                   = "512"
  execution_role_arn       = aws_iam_role.ecs_exec_role.arn

  container_definitions = jsonencode([{
    name  = "web-container"
    image = "nginx:latest"
    portMappings = [{
      containerPort = 80
      hostPort      = 80
    }]
```

```
  }])
}
```

Category: Serverless APIs & Logic (Scripts 26–30)

26. API Gateway (REST) with Lambda Proxy Integration

```
# The REST API Container
resource "aws_api_gateway_rest_api" "api" {
  name        = "OrderProcessingAPI"
  description = "Serverless API for Order Management"
}

# The Resource Path (e.g., /orders)
resource "aws_api_gateway_resource" "orders" {
  rest_api_id = aws_api_gateway_rest_api.api.id
  parent_id   = aws_api_gateway_rest_api.api.root_resource_id
  path_part   = "orders"
}

# The HTTP Method (POST)
resource "aws_api_gateway_method" "post_order" {
  rest_api_id   = aws_api_gateway_rest_api.api.id
  resource_id   = aws_api_gateway_resource.orders.id
  http_method   = "POST"
  authorization = "NONE"
}

# The Integration (Lambda Proxy)
resource "aws_api_gateway_integration" "lambda_int" {
  rest_api_id = aws_api_gateway_rest_api.api.id
  resource_id = aws_api_gateway_resource.orders.id
  http_method = aws_api_gateway_method.post_order.http_method

  integration_http_method = "POST"
  type                    = "AWS_PROXY" # Critical: Passes the whole
request to Lambda
  uri                     = aws_lambda_function.order_processor.invoke_arn
}
```

27. Lambda Function with VPC Access

```
resource "aws_lambda_function" "order_processor" {
  function_name = "ProcessOrderFunction"
  role          = aws_iam_role.lambda_exec.arn
  handler       = "index.handler"
  runtime       = "python3.11"
  filename      = "function.zip"

  # Networking: Connecting to VPC Resources
  vpc_config {
    subnet_ids         = [aws_subnet.private_1.id, aws_subnet.private_2.id]
    security_group_ids = [aws_security_group.lambda_sg.id]
  }

  environment {
    variables = {
      DB_TABLE = aws_dynamodb_table.users.name
```

```
    }
  }
}
```

28. Lambda Permission for API Gateway

```
resource "aws_lambda_permission" "apigw_lambda" {
  statement_id  = "AllowExecutionFromAPIGateway"
  action        = "lambda:InvokeFunction"
  function_name = aws_lambda_function.order_processor.function_name
  principal     = "apigateway.amazonaws.com"

  # Restrict to a specific API to prevent cross-account attacks
  source_arn = "${aws_api_gateway_rest_api.api.execution_arn}/*/*"
}
```

Category: Configuration & Secrets (Scripts 31–35)

31. SSM Parameter Store (Hierarchical)

```
resource "aws_ssm_parameter" "db_url" {
  name        = "/prod/orders/db_url"
  description = "Database Endpoint for Orders Service"
  type        = "String"
  value       = aws_rds_cluster.aurora.endpoint
}

resource "aws_ssm_parameter" "api_key" {
  name        = "/prod/orders/external_api_key"
  description = "Third party API key"
  type        = "SecureString" # Encrypted with KMS automatically
  value       = "secret-value-123"
}
```

32. Secrets Manager with RDS Integration

This script shows how Secrets Manager can manage the primary database credentials and automatically update them.

```
resource "aws_secretsmanager_secret" "db_creds" {
  name       = "orders-db-credentials-v2"
  kms_key_id = aws_kms_key.app_key.id
}

resource "aws_secretsmanager_secret_version" "db_creds_val" {
  secret_id = aws_secretsmanager_secret.db_creds.id
  secret_string = jsonencode({
    username             = "admin"
    password             = "P@ssw0rd2026!"
    engine               = "mysql"
    host                 = aws_rds_cluster.aurora.endpoint
    port                 = 3306
    dbClusterIdentifier  = aws_rds_cluster.aurora.cluster_identifier
  })
```

```
}
```

Category: Event-Driven Architecture (Scripts 36–40)

36. SNS Topic to SQS Subscription (The Fan-Out)

```
# 1. Create the SNS Topic (The Broadcaster)
resource "aws_sns_topic" "order_events" {
  name = "order-events-topic"
}

# 2. Create the SQS Queue (The Consumer)
resource "aws_sqs_queue" "shipping_queue" {
  name = "shipping-processing-queue"
}

# 3. Subscribe the Queue to the Topic
resource "aws_sns_topic_subscription" "sns_to_sqs" {
  topic_arn = aws_sns_topic.order_events.arn
  protocol  = "sqs"
  endpoint  = aws_sqs_queue.shipping_queue.arn
}

# 4. Policy to allow SNS to write to SQS
resource "aws_sqs_queue_policy" "allow_sns" {
  queue_url = aws_sqs_queue.shipping_queue.id
  policy = jsonencode({
    Statement = [{
      Effect    = "Allow"
      Principal = { Service = "sns.amazonaws.com" }
      Action    = "sqs:SendMessage"
      Resource  = aws_sqs_queue.shipping_queue.arn
      Condition = { ArnEquals = { "aws:SourceArn" =
aws_sns_topic.order_events.arn } }
    }]
  })
}
```

Category: Global Traffic Management (Scripts 41–44)

41. Route 53 Weighted Routing Policy

Used for **Blue/Green deployments** or **A/B Testing**. This allows you to send 10% of traffic to a "New" version and 90% to the "Stable" version.

```
resource "aws_route53_record" "blue_version" {
  zone_id = aws_route53_zone.main.zone_id
  name    = "app.ssdn-labs.com"
  type    = "A"
  ttl     = "60"
```

```
  weighted_routing_policy {
    weight = 90 # 90% of traffic
  }

  set_identifier = "stable-v1"
  records        = ["1.2.3.4"]
}

resource "aws_route53_record" "green_version" {
  zone_id = aws_route53_zone.main.zone_id
  name    = "app.ssdn-labs.com"
  type    = "A"
  ttl     = "60"

  weighted_routing_policy {
    weight = 10 # 10% of traffic
  }

  set_identifier = "new-v2"
  records        = ["5.6.7.8"]
}
```

42. Route 53 Latency-Based Routing

Automatically routes users to the AWS region that provides the lowest latency for their specific location.

```
resource "aws_route53_record" "latency_record" {
  zone_id = aws_route53_zone.main.zone_id
  name    = "global-api.ssdn-labs.com"
  type    = "A"

  latency_routing_policy {
    region = "us-east-1"
  }

  set_identifier = "us-east-endpoint"
  alias {
    name                   = aws_lb.us_east_alb.dns_name
    zone_id                = aws_lb.us_east_alb.zone_id
    evaluate_target_health = true
  }
}
```

Category: Real-Time Data Streaming (Scripts 45–47)

45. Kinesis Data Stream (Sharded)

```
resource "aws_kinesis_stream" "ingest_stream" {
  name             = "financial-tx-stream"
  shard_count      = 3 # Scales capacity
  retention_period = 24 # Hours data stays in stream

  shard_level_metrics = [
    "IncomingBytes",
```

```
    "OutgoingBytes",
  ]

  stream_mode_details {
    stream_mode = "PROVISIONED"
  }
}
```

46. Kinesis Firehose to S3 (Data Lake Ingest)

This is the "Zero Code" way to move streaming data into an S3 Data Lake for long-term storage or Athena analysis.

```
resource "aws_kinesis_firehose_delivery_stream" "s3_archive" {
  name        = "kinesis-firehose-s3-backup"
  destination = "extended_s3"

  extended_s3_configuration {
    role_arn   = aws_iam_role.firehose_role.arn
    bucket_arn = aws_s3_bucket.data_lake.arn

    # Buffer data to save costs/IOPS
    buffer_size = 5 # MB
    buffer_interval = 300 # Seconds
  }
}
```

Category: Identity & Governance (Scripts 48–50)

48. IAM OIDC Identity Provider (Okta Integration)

This allows users to log into the AWS Console using their Okta credentials instead of having local IAM users.

```
resource "aws_iam_openid_connect_provider" "okta" {
  url = "https://dev-12345.okta.com"

  client_id_list = [
    "sts.amazonaws.com",
  ]

  # Fingerprint varies by region/provider - essential for trust
  thumbprint_list = ["9e99a48a9960b14926bb7f3b02e22da2b0ab7280"]
}
```

49. IAM Role for Federated Users (Trust Policy)

The "Glue" between Okta and AWS. This defines who from Okta can assume which permissions in AWS.

```
resource "aws_iam_role" "federated_admin" {
  name = "Okta-Admin-Role"
```

```
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRoleWithWebIdentity"
        Effect = "Allow"
        Principal = { Federated = aws_iam_openid_connect_provider.okta.arn
}
        Condition = {
          StringEquals = {
            "dev-12345.okta.com:aud": "sts.amazonaws.com"
          }
        }
      }
    ]
  })
}
```

50. Organizations: Service Control Policy (SCP)

The "Super-Admin" layer. This script can prevent *anyone* (even a root user) in a sub-account from deleting CloudTrail logs or using unauthorized regions.

```
resource "aws_organizations_policy" "restrict_regions" {
  name        = "DenyNonUSRegions"
  description = "Compliance: Only allow resources in US-East-1 and US-West-
2"
  type        = "SERVICE_CONTROL_POLICY"

  content = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid    = "DenyAllOutsideUS"
        Effect = "Deny"
        NotAction = "iam:*"
        Resource = "*"
        Condition = {
          StringNotEquals = {
            "aws:RequestedRegion": ["us-east-1", "us-west-2"]
          }
        }
      }
    ]
  })
}
```

Category: Global Traffic Management (Scripts 41–44)

41. Route 53 Weighted Routing Policy

Used for **Blue/Green deployments** or **A/B Testing**. This allows you to send 10% of traffic to a "New" version and 90% to the "Stable" version.

```
resource "aws_route53_record" "blue_version" {
  zone_id = aws_route53_zone.main.zone_id
  name    = "app.ssdn-labs.com"
```

```
  type    = "A"
  ttl     = "60"

  weighted_routing_policy {
    weight = 90 # 90% of traffic
  }

  set_identifier = "stable-v1"
  records        = ["1.2.3.4"]
}

resource "aws_route53_record" "green_version" {
  zone_id = aws_route53_zone.main.zone_id
  name    = "app.ssdn-labs.com"
  type    = "A"
  ttl     = "60"

  weighted_routing_policy {
    weight = 10 # 10% of traffic
  }

  set_identifier = "new-v2"
  records        = ["5.6.7.8"]
}
```

42. Route 53 Latency-Based Routing

Automatically routes users to the AWS region that provides the lowest latency for their specific location.

```
resource "aws_route53_record" "latency_record" {
  zone_id = aws_route53_zone.main.zone_id
  name    = "global-api.ssdn-labs.com"
  type    = "A"

  latency_routing_policy {
    region = "us-east-1"
  }

  set_identifier = "us-east-endpoint"
  alias {
    name                   = aws_lb.us_east_alb.dns_name
    zone_id                = aws_lb.us_east_alb.zone_id
    evaluate_target_health = true
  }
}
```

---

43. App Runner (Serverless Web Apps)

App Runner is the direct equivalent: source code to URL without managing a single server.

```
resource "aws_apprunner_service" "web_app" {
  service_name = "serverless-python-app"

  source_configuration {
```

```
    image_repository {
      image_identifier      =
"${aws_ecr_repository.app.repository_url}:latest"
      image_repository_type = "ECR"
    }
    auto_deployments_enabled = true
  }
}
```

Category: AI & Machine Learning Infrastructure

44. Amazon Bedrock Agent & Knowledge Base

It provisions a Generative AI agent that can query an S3 bucket (RAG) to answer user questions.

```
resource "aws_bedrockagent_agent" "helper" {
  agent_name              = "it-support-agent"
  foundation_model        = "anthropic.claude-3-sonnet"
  instruction             = "You are a helpful IT support assistant."
  agent_resource_role_arn = aws_iam_role.bedrock_role.arn
}

# The Knowledge Base (RAG)
resource "aws_bedrockagent_knowledge_base" "docs" {
  name     = "company-policies"
  role_arn = aws_iam_role.bedrock_role.arn

  knowledge_base_configuration {
    type = "VECTOR"
    vector_knowledge_base_configuration {
      embedding_model_arn = "arn:aws:bedrock:us-east-1::foundation-
model/amazon.titan-embed-text-v2"
    }
  }
}
```

45.SageMaker Endpoints (Real-time Inference)

Deploying a custom-trained model for real-time predictions.

```
resource "aws_sagemaker_endpoint_configuration" "config" {
  name = "model-v1-config"
  production_variants {
    variant_name           = "all-traffic"
    model_name             = aws_sagemaker_model.trained_model.name
    initial_instance_count = 1
    instance_type          = "ml.t2.medium"
  }
}
```

46. Lake Formation (Data Governance)

Instead of messy IAM policies for S3, Lake Formation provides a **SQL-like permission model** for your Data Lake.

```
resource "aws_lakeformation_permissions" "data_analyst" {
  principal = aws_iam_role.analyst_role.arn
  permissions = ["SELECT"]

  table {
    database_name = "sales_db"
    name          = "transactions"
  }
}
```

Category: Enterprise Security & Governance

47. AWS WAFv2 with Managed Rule Groups

Web Application Firewall (WAF) is the primary defense against SQL Injection and Cross-Site Scripting (XSS). In 2026, the best practice is to use **AWS Managed Rules** for automatic updates.

```
resource "aws_wafv2_web_acl" "main" {
  name        = "enterprise-waf"
  scope       = "REGIONAL" # Use "CLOUDFRONT" for global distros
  description = "WAF with Managed Rules and Rate Limiting"

  default_action { allow {} }

  # 1. AWS Managed Core Rule Set (Common vulnerabilities)
  rule {
    name     = "aws-common-rules"
    priority = 10
    override_action { none {} }
    statement {
      managed_rule_group_statement {
        name        = "AWSManagedRulesCommonRuleSet"
        vendor_name = "AWS"
      }
    }
    visibility_config {
      cloudwatch_metrics_enabled = true
      metric_name                = "CommonRules"
      sampled_requests_enabled   = true
    }
  }

  # 2. Custom Rate Limiting (Prevent Brute Force)
  rule {
    name     = "rate-limit"
    priority = 20
    action { block {} }
    statement {
      rate_based_statement {
        limit              = 2000
        aggregate_key_type = "IP"
```

```
      }
    }
    visibility_config {
      cloudwatch_metrics_enabled = true
      metric_name                = "RateLimit"
      sampled_requests_enabled   = true
    }
  }

  visibility_config {
    cloudwatch_metrics_enabled = true
    metric_name                = "WAFMain"
    sampled_requests_enabled   = true
  }
}
```

48. CloudTrail for Audit & Compliance

A mandatory setup for any enterprise. This script ensures every single API call in the account is logged to a secure S3 bucket.

```
resource "aws_cloudtrail" "main" {
  name                          = "management-events-trail"
  s3_bucket_name                = aws_s3_bucket.trail_logs.id
  include_global_service_events = true
  is_multi_region_trail         = true
  enable_log_file_validation    = true # Ensures logs aren't tampered with
}
```

Category: Hybrid Networking

49. VPC Peering (Inter-VPC Connectivity)

VPC Peering allows two private networks to talk securely over the AWS backbone.

```
resource "aws_vpc_peering_connection" "app_to_db" {
  peer_vpc_id = aws_vpc.db_vpc.id
  vpc_id      = aws_vpc.app_vpc.id
  auto_accept = true

  tags = { Name = "App-to-DB-Peering" }
}

# Important: You must update routes in BOTH VPCs manually
resource "aws_route" "peering_route" {
  route_table_id            = aws_route_table.app_rt.id
  destination_cidr_block    = "10.1.0.0/16" # DB VPC Range
  vpc_peering_connection_id = aws_vpc_peering_connection.app_to_db.id
}
```

50. AWS Transfer Family (Managed SFTP)

Enterprises still use SFTP for legacy integrations. This script provisions a managed SFTP endpoint backed by S3.

```
resource "aws_transfer_server" "sftp" {
  identity_provider_type = "SERVICE_MANAGED"
  endpoint_type          = "PUBLIC"
  protocols              = ["SFTP"]

  tags = { Name = "Partner-Data-Exchange" }
}
```

Category: Workflow Orchestration

51. Step Functions "Standard" State Machine

The "Brain" of your system. Use this to **Distributed Transactions** (The Saga Pattern).

```
resource "aws_sfn_state_machine" "order_flow" {
  name     = "OrderProcessingWorkflow"
  role_arn = aws_iam_role.sfn_role.arn

  definition = jsonencode({
    StartAt = "ProcessPayment"
    States = {
      ProcessPayment = {
        Type     = "Task"
        Resource = aws_lambda_function.pay_lambda.arn
        Next     = "ShipOrder"
      }
      ShipOrder = {
        Type     = "Task"
        Resource = aws_lambda_function.ship_lambda.arn
        End      = true
      }
    }
  })
}
```

52. EventBridge Scheduled Rule (Cloud Cron)

For running automated cleanup or report generation jobs without an external scheduler.

```
resource "aws_cloudwatch_event_rule" "nightly_report" {
  name                = "nightly-db-cleanup"
  schedule_expression = "cron(0 1 * * ? *)" # 1 AM daily
}

resource "aws_cloudwatch_event_target" "trigger_lambda" {
  rule      = aws_cloudwatch_event_rule.nightly_report.name
  target_id = "CleanupLambda"
  arn       = aws_lambda_function.cleanup.arn
}
```

53. Resource Tagging (Governance Policy)

The "Glue" for cost allocation. **Tags are not optional** in production.

```
# This is a conceptual best practice implemented via locals
locals {
  common_tags = {
    Project     = "SSDN-Internal"
    Environment = "Production"
    ManagedBy   = "Terraform"
    Owner       = "IT-Ops"
  }
}

# Example usage on any resource:
# tags = local.common_tags
```

---

54. AWS Control Tower: Disallowing Public S3 via Guardrail

This script enables a mandatory Control Tower guardrail at the Organizational Unit (OU) level using the `aws_controltower_control` resource.

```
resource "aws_organizations_organizational_unit" "workloads" {
  name      = "Workloads"
  parent_id = data.aws_organizations_organization.main.roots[0].id
}

# Enforce "No Public S3" across the entire OU
resource "aws_controltower_control" "disallow_public_s3" {
  control_identifier = "arn:aws:controltower:us-east-1::control/AWS-
GR_S3_ACCOUNT_LEVEL_PUBLIC_ACCESS_BLOCKS_PERIODIC"
  target_identifier  = aws_organizations_organizational_unit.workloads.arn
}
```

55. Multi-Region IP Address Manager (IPAM)

Large enterprises struggle with IP overlap. IPAM automates IP allocation across regions and accounts.

```
resource "aws_vpc_ipam" "main" {
  operating_regions { region_name = "us-east-1" }
  operating_regions { region_name = "eu-west-1" }
}

resource "aws_vpc_ipam_pool" "corporate" {
  address_family = "ipv4"
  ipam_scope_id  = aws_vpc_ipam.main.private_default_scope_id
  locale         = "us-east-1"
}
```

Category: Modern Data Mesh & Networking

56. AWS PrivateLink Endpoint Service

It Explains how to expose an internal service to *other* AWS accounts privately without VPC Peering or Transit Gateway.

```
# The Service Provider side
resource "aws_vpc_endpoint_service" "app_service" {
  acceptance_required        = true
  network_load_balancer_arns = [aws_lb.internal_nlb.arn]

  tags = { Name = "Shared-Service-Provider" }
}
```

57. Lake Formation Data Permissions

Moving away from complex IAM policies for data. This script grants SQL-like `SELECT` permissions on specific Glue tables.

```
resource "aws_lakeformation_permissions" "analyst_access" {
  principal = aws_iam_role.data_analyst.arn
  permissions = ["SELECT"]

  table {
    database_name = "sales_db"
    name          = "transactions"
  }
}
```

58. App Mesh: Virtual Service & Router

For 2026 microservices, networking logic (retries, timeouts) should be moved to the Service Mesh.

```
resource "aws_appmesh_mesh" "main" {
  name = "business-logic-mesh"
}

resource "aws_appmesh_virtual_router" "service_a_router" {
  name      = "service-a-router"
  mesh_name = aws_appmesh_mesh.main.name
  spec {
    listener {
      port_mapping { port = 8080; protocol = "http" }
    }
  }
}
```

Category: Generative AI Infrastructure

59. Amazon Bedrock Knowledge Base (RAG)

It sets up an AI "Knowledge Base" that connects an S3 bucket to an OpenSearch vector index.

```
resource "aws_bedrockagent_knowledge_base" "company_kb" {
  name     = "internal-docs-kb"
  role_arn = aws_iam_role.bedrock_role.arn

  knowledge_base_configuration {
    type = "VECTOR"
    vector_knowledge_base_configuration {
      embedding_model_arn = "arn:aws:bedrock:us-east-1::foundation-
model/amazon.titan-embed-text-v2"
    }
  }

  storage_configuration {
    type = "OPENSEARCH_SERVERLESS"
    opensearch_serverless_configuration {
      collection_arn    =
aws_opensearchserverless_collection.vector_store.arn
      vector_index_name = "bedrock-knowledge-base-default-index"
      field_mapping {
        vector_field   = "bedrock-knowledge-base-default-vector"
        text_field     = "AMAZON_BEDROCK_TEXT_CHUNK"
        metadata_field = "AMAZON_BEDROCK_METADATA"
      }
    }
  }
}
```

60. Bedrock Agent with Action Groups

Define an AI agent that can actually *take actions* (like triggering a Lambda function) rather than just chatting.

```
resource "aws_bedrockagent_agent" "it_bot" {
  agent_name              = "it-automation-agent"
  foundation_model        = "anthropic.claude-3-sonnet"
  instruction             = "You are an assistant that helps reset
passwords via Lambda."
  agent_resource_role_arn = aws_iam_role.bedrock_role.arn
}
```

61. SageMaker Serverless Inference Endpoint

For deploying ML models without managing underlying EC2 instances.

```
resource "aws_sagemaker_endpoint_configuration" "serverless" {
  name = "model-v1-serverless-config"

  production_variants {
    variant_name           = "AllTraffic"
    model_name             = aws_sagemaker_model.main.name

    serverless_config {
      max_concurrency   = 10
      memory_size_in_mb = 2048
    }
```

```
  }
}
```

Category: Hybrid Networking & Edge

62. Transit Gateway with Peering

Transit Gateway (TGW) is the "Core Router." This script sets up a hub-and-spoke model.

```
# The Central Router
resource "aws_ec2_transit_gateway" "network_hub" {
  description = "Enterprise Core Router"
  auto_accept_shared_attachments = "enable"

  tags = { Name = "Main-TGW" }
}

# Attaching a Spoke VPC to the Hub
resource "aws_ec2_transit_gateway_vpc_attachment" "spoke_a" {
  subnet_ids         = [aws_subnet.private_1.id]
  transit_gateway_id = aws_ec2_transit_gateway.network_hub.id
  vpc_id             = aws_vpc.spoke_vpc.id
}
```

63. Global Accelerator (Fixed IP Entry)

This provides two static anycast IP addresses that route users to the nearest healthy endpoint.

```
resource "aws_globalaccelerator_accelerator" "main" {
  name            = "GlobalAppEntry"
  ip_address_type = "IPV4"
  enabled         = true
}

resource "aws_globalaccelerator_listener" "http" {
  accelerator_arn = aws_globalaccelerator_accelerator.main.id
  port_range {
    from_port = 80
    to_port   = 80
  }
  protocol = "TCP"
}
```

Category: Resilience & Orchestration

64. Step Functions (The State Machine)

This is the "Director" for Lambda functions. It handles retries, error branching, and parallel processing without "spaghetti code."

```
resource "aws_sfn_state_machine" "order_process" {
  name     = "Order-Orchestrator"
  role_arn = aws_iam_role.sfn_role.arn

  definition = jsonencode({
    StartAt = "ProcessPayment",
    States = {
      ProcessPayment = {
        Type     = "Task",
        Resource = aws_lambda_function.payment_lambda.arn,
        Next     = "ShipOrder"
      },
      ShipOrder = {
        Type = "Task",
        Resource = aws_lambda_function.shipping_lambda.arn,
        End = true
      }
    }
  })
}
```

65. Lambda Event Source Mapping (SQS to Lambda)

Connects a queue to a function so the function only triggers when there is work to do.

```
resource "aws_lambda_event_source_mapping" "sqs_trigger" {
  event_source_arn = aws_sqs_queue.orders_queue.arn
  function_name    = aws_lambda_function.worker.arn
  batch_size       = 10 # Process 10 messages at once
}
```

Category: Advanced Governance

66. AWS Budget Alerts

This script sends an SNS alert when the monthly spend exceeds **$100**.

```
resource "aws_budgets_budget" "monthly_limit" {
  name              = "monthly-budget-alert"
  budget_type       = "COST"
  limit_amount      = "100"
  limit_unit        = "USD"
  time_unit         = "MONTHLY"

  notification {
    comparison_operator        = "GREATER_THAN"
    threshold                  = 80
    threshold_type             = "PERCENTAGE"
    notification_type          = "ACTUAL"
    subscriber_email_addresses = ["manager@ssdn-labs.com"]
```

```
  }
}
```

67. Service Control Policy (SCP) - Deny Root User

Used at the Organization level to prevent even the Root user of a sub-account from performing dangerous actions.

```
resource "aws_organizations_policy" "deny_root" {
  name    = "RestrictRootUser"
  content = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Sid      = "DenyRootUsage"
      Effect   = "Deny"
      Action   = "*"
      Resource = "*"
      Condition = { StringLike = { "aws:PrincipalArn" =
["arn:aws:iam::*:root"] } }
    }]
  })
}
```

68. Config Rule: Ensure All EBS Volumes Encrypted

Automated compliance. If a developer creates an unencrypted disk, AWS Config will flag it.

```
resource "aws_config_config_rule" "ebs_encryption" {
  name = "encrypted-volumes"
  source {
    owner             = "AWS"
    source_identifier = "ENCRYPTED_VOLUMES"
  }
}
```